DIGITAL LOGIC & COMPUTER DESIGN
DETAILED QUESTION BANK

5 MARKS • 8 MARKS • 10 MARKS
8 Uploaded PPTs • Exam-Oriented Answers + Numericals

Each PPT contains 2 × 5-mark, 2 × 8-mark and 2 × 10-mark questions. Answers are expanded according to marks and include numerical problems where supported by the slides.

PPT	5M	8M	10M
1. Introduction	2	2	2
2. Assembly Language	2	2	2
3. Machine Language	2	2	2
4. Programming	2	2	2
5. Addressing Modes	2	2	2
6. Parallelism	2	2	2
7. Memory Arrays	2	2	2
8. Sequential Building Blocks	2	2	2

1. Introduction

Q1. [5 Marks] Define Computer Architecture and explain what it specifies.

Answer:
Computer Architecture is the programmer's view of a computer. It describes the interface through which software communicates with hardware.
The PPT identifies the following major specifications:
1. Instruction Set Architecture (ISA) – the instructions understood by the processor.
2. Operand locations – where operands are stored, such as registers and memory.
3. Data representation – how data values are represented.
4. Instruction formats – how instructions are organized/encoded.
Thus, architecture provides the interface that determines how a program is executed by the processor.

Q2. [5 Marks] Why is Computer Architecture studied? Explain its importance.

Answer:
Computer Architecture is important because it:
1. Bridges the gap between hardware and software.
2. Explains how programs are executed by a processor.
3. Helps in designing efficient and high-performance computer systems.
4. Helps maintain compatibility between software and hardware.
5. Provides a foundation for understanding processor organization and programming.
Therefore, it connects what a programmer writes with what the hardware actually executes.

Q3. [8 Marks] Explain Instruction Set Architecture (ISA) and the idea that instruction sets are like computer 'dialects'.

Answer:
Instruction Set Architecture is the programmer-visible interface of a processor. It defines the operations/instructions that the processor understands, operand locations, data representation and instruction formats.
The PPT points out that many architectures exist, such as x86, MIPS, SPARC and PowerPC. Each architecture defines its own instruction set.
Instruction sets can be compared to different dialects of a language: the basic purpose is similar, but the exact words and rules differ. Once one instruction set is understood, learning another is easier because many basic operations—such as addition, subtraction and jumps—are common in concept.
Hence, ISA is the key boundary between software and processor hardware.

Q4. [8 Marks] Explain the role of instructions in the execution of computer programs.

Answer:
Instructions are the basic words of a computer's language. Even complex applications are ultimately translated into simple machine-level operations.
The execution process can be understood as:
1. A programmer writes an application.
2. The program is translated/compiled into instructions supported by the target architecture.
3. The processor fetches and interprets these instructions.
4. Instructions perform operations such as arithmetic, logical operations, memory access and control flow.
5. The processor updates registers/memory and continues with the next instruction.
Thus, instructions provide the mechanism through which software controls the hardware.

Q5. [10 Marks] Explain the major layers from software to hardware and how Computer Architecture connects them.

Answer:
A computer system can be viewed as a set of layers. At the top is application software, which expresses the user's task. The operating system manages resources and provides services. Computer architecture provides the programmer-visible interface, including the ISA, operand locations, data representation and instruction formats. Below architecture are lower hardware implementation levels such as microarchitecture, logic and digital circuits.
Computer architecture is therefore the important boundary between software and hardware. A program does not directly manipulate transistors; instead, it uses the instructions and data conventions defined by the architecture. The

processor's implementation then carries out those instructions.
This layered view is useful because it separates what software expects from how the hardware implements it. Different architectures can implement similar software goals using different instruction sets and hardware organizations.

Q6. [10 Marks] Discuss why many computer architectures exist and compare their instruction-set concept using examples from the PPT.

Answer:
Different architectures exist because processors can be designed with different instruction sets and design goals. The PPT gives examples including x86, MIPS, SPARC and PowerPC.
An architecture defines:
1. The instruction set available to programs.
2. The locations from which operands can be obtained.
3. Data representation.
4. Instruction formats.
5. The programmer-visible behavior of the processor.
For example, a program targeting MIPS uses MIPS instructions, while an x86 program uses the x86 instruction set. Although the exact instructions differ, many basic operations have the same purpose, such as add, subtract and jump. Therefore, architectures can be thought of as different computer-language dialects: different instruction vocabularies and formats, but common computational concepts.

2. Assembly Language

Q1. [5 Marks] What is an operand? Explain the three operand locations in MIPS with an example.

Answer:
An operand is the data used by an instruction. The PPT identifies three operand locations in MIPS:
1. Registers – high-speed storage inside the CPU.
2. Memory – large storage for program instructions and data.
3. Constants/immediates – fixed values encoded directly in the instruction.
Example:
addi $t0, $t1, 10
Here $t0 is the destination register, $t1 is the source register, and 10 is the immediate constant.
Thus, an instruction can obtain its data from registers, memory or directly from an encoded constant.

Q2. [5 Marks] Explain immediate operands and immediate instructions in MIPS.

Answer:
An immediate operand is a constant embedded directly in an instruction. MIPS uses a 16-bit signed two's-complement field for immediate values.
Advantages:
1. The constant is immediately available.
2. A separate memory access is not needed for a small constant.
3. It can reduce the number of instructions required.
Examples include:
addi – add an immediate constant.
andi – AND with an immediate.
ori – OR with an immediate.
xori – XOR with an immediate.
Therefore, immediate operands are useful for small fixed values.

Q3. [8 Marks] Compare registers, memory and constants/immediates as operand locations.

Answer:
Registers are high-speed storage locations inside the CPU. They are useful for frequently accessed data and intermediate results, but their number is limited.
Memory provides much larger storage and contains program instructions and data that cannot fit in registers. However, memory access is slower than register access.
Constants/immediates are fixed values encoded directly within an instruction. They are convenient for small constants because the processor does not need to load them from memory.

For example, addi $t0,$t1,10 uses an immediate. If a value is stored in memory, lw is used to load it into a register. If two values are already in registers, an instruction such as add can operate directly on them.
Thus, registers emphasize speed, memory capacity, and immediates convenient constant access.

Q4. [8 Marks] Explain the role of lw and sw instructions in MIPS memory access.

Answer:
MIPS has only a limited number of registers, so program data and instructions are also stored in main memory. The PPT emphasizes that memory is larger but slower than registers.
The two basic data-transfer instructions are:
• lw (load word): transfers a word from memory into a register.
• sw (store word): transfers a word from a register into memory.
A memory address is formed using a base address and an offset. A typical sequence is:
1. Keep the base address in a register.
2. Calculate the required effective address.
3. Use lw to bring the value into a register.
4. Perform arithmetic/logic operations on registers.
5. Use sw if the result must be written back to memory.
This is central to the MIPS load/store architecture.

Q5. [10 Marks] Explain how an arithmetic expression involving registers, constants and memory can be implemented in MIPS.

Answer:
MIPS performs arithmetic primarily on register operands. Suppose values are needed from memory and a constant must be added.
Step 1: Load the memory value into a register using lw.
Step 2: If another operand is already in a register, use it directly.
Step 3: Use add or sub for register-register arithmetic.
Step 4: Use addi when one operand is a suitable constant.
Step 5: Store the result using sw if it must be written to memory.
For example, the operation result = memory_value + register_value + 10 can conceptually be implemented as:

```
lw $t0, offset($s0)
add $t1, $t0, $s1
addi $t1, $t1, 10
sw $t1, offset2($s0)
```

The important principle is that memory values are moved into registers before arithmetic is performed.

Q6. [10 Marks] Explain why both registers and memory are necessary in MIPS.

Answer:
Registers and memory serve different purposes.
Registers:
• Are located inside the CPU.
• Provide very fast access.
• Hold frequently used operands, addresses and intermediate results.
• Are limited in number.
Memory:
• Has much larger capacity.
• Stores program instructions and large amounts of data.
• Is slower to access than registers.
• Is accessed using load/store instructions.
Because registers cannot contain all program data, memory is necessary for capacity. Because memory is slower, registers are necessary for efficient execution of frequently used values.
MIPS therefore follows a register-based approach: data is brought from memory into registers with lw, processed using arithmetic/logic instructions, and written back with sw when necessary.

3. Machine Language

Q1. [5 Marks] Explain R-type, I-type and J-type instruction formats.

Answer:
MIPS instructions are 32-bit binary patterns. The three main formats are:
1. R-type – mainly register-register operations.
2. I-type – immediate operations, load/store operations and branches.
3. J-type – jump instructions.
The processor first examines the opcode and then interprets the remaining fields according to the selected format.

Q2. [5 Marks] Explain the fields of an R-type instruction.

Answer:
An R-type instruction contains:
• op – opcode; for R-type it is 000000.
• rs – first source register.
• rt – second source register.
• rd – destination register.
• shamt – shift amount; normally 0 except for shift instructions.
• funct – selects the exact operation.
For example, in add $s0,$s1,$s2, $s1 and $s2 are sources and $s0 is the destination.

Q3. [8 Marks] Explain the three MIPS instruction formats with their important fields.

Answer:
R-type:
Used mainly for register-register ALU operations. It contains rs, rt, rd, shamt and funct fields in addition to the opcode.
I-type:
Used for immediate operations, loads/stores and branches. It contains rs, rt and a 16-bit immediate/offset field.
J-type:
Used for jump instructions. It contains a 26-bit address field.
A useful decoding order is:
opcode → determine format → interpret fields → determine the final assembly instruction.
All three formats fit into the fixed 32-bit instruction size.

Q4. [8 Marks] Explain how a 32-bit MIPS machine instruction is interpreted by the processor.

Answer:
A MIPS instruction is stored in memory as a 32-bit binary pattern. The processor does not directly see the assembly mnemonic such as add or lw. Instead, it sees bit fields.
The interpretation process is:
1. Fetch the 32-bit instruction.
2. Read the opcode.
3. Use the opcode to determine the instruction format and operation category.
4. Interpret the remaining bits as register numbers, an immediate/offset, shift amount, function code or jump address.
5. Perform the operation specified by those fields.
For example, an opcode corresponding to addi tells the processor to interpret the remaining fields as rs, rt and a 16-bit immediate.

Q5. [10 Marks] Decode the instruction 'add $s0, $s1, $s2' and explain the function of every field.

Answer:
The instruction is an R-type instruction because it performs a register-register ALU operation.
For:
add $s0, $s1, $s2
the fields are:
• opcode = 000000, identifying R-type.
• rs = $s1, the first source register.
• rt = $s2, the second source register.
• rd = $s0, the destination register.
• shamt = 0 because add is not a shift instruction.
• funct = the function code that selects the add operation.

During execution, the processor reads the values from $s1 and $s2, performs addition, and writes the result into $s0. The important point is that assembly order and machine-field order are not identical: the destination appears first in assembly, but is encoded in the rd field.

Q6. [10 Marks] Explain logical and shift instructions in MIPS machine language, including logical, arithmetic and immediate shifts.

Answer:
MIPS provides logical operations such as AND, OR, XOR and NOR. These operate bit-by-bit. Logical immediate versions include andi, ori and xori.
Shift instructions include:
• sll – shift left logical.
• srl – shift right logical.
• sra – shift right arithmetic.
The shift amount is represented by the shamt field for these R-type shift instructions.
For a left shift, zeros enter the least significant bit positions. For a logical right shift, zeros enter the most significant positions. For an arithmetic right shift, the sign bit is copied into the most significant positions.
Thus, logical operations manipulate individual bits, while shifts move bit patterns and can preserve or change the sign depending on the type of right shift.

4. Programming

Q1. [5 Marks] Explain MIPS arithmetic and logical instructions with examples.

Answer:
MIPS provides arithmetic instructions such as add, sub and addi.
Examples:
add $t0,$s1,$s2 → $t0 = $s1 + $s2
sub $t0,$t3,$t5 → $t0 = $t3 – $t5
addi $t0,$s3,-12 → $t0 = $s3 – 12
MIPS also provides multiplication and division instructions using the special hi and lo registers.
Logical instructions include AND, OR, XOR and NOR and operate bit-by-bit.

Q2. [5 Marks] What is the purpose of branching and conditional statements in MIPS?

Answer:
Branching changes the normal sequential flow of a program. It is required for decisions and repetition.
Conditional statements such as if and if-else are implemented using conditional branches. A branch checks a condition and transfers control to a target label when the condition is satisfied.
Thus, branching provides control flow, while arithmetic and logical instructions perform the actual data processing.

Q3. [8 Marks] Explain the four basic logical operations in MIPS and their uses.

Answer:
MIPS provides AND, OR, XOR and NOR.
AND: operates bit-by-bit and is useful for masking selected bits. AND can force selected bits to zero.
OR: combines bits from two operands and can be used to set selected bits.
XOR: produces 1 when corresponding bits differ and can be used for bit toggling or comparison of bit patterns.
NOR: produces the complement of OR. Since $0 contains all zeros, NOT A can be implemented as A NOR $0.
The PPT also gives hexadecimal examples demonstrating the bit-by-bit results.

Q4. [8 Marks] Explain how loops are implemented using labels, updates and branch instructions.

Answer:
A loop repeatedly executes a group of instructions. In MIPS, the structure generally contains:
1. A label marking the beginning of the loop.
2. Instructions that perform the required work.
3. An update to a loop variable or counter.
4. A condition check.
5. A conditional branch back to the label when the loop must continue.

For example, a counter can be incremented using addi and then compared using a branch instruction. The branch determines whether execution returns to the loop label or continues to the next part of the program.
Thus, loops are built from ordinary instructions plus control-flow branches.

Q5. [10 Marks] Explain how an if-else statement is implemented in MIPS assembly.

Answer:
An if-else statement requires two possible paths.
General structure:
1. Evaluate the condition using registers.
2. Use a conditional branch to select the appropriate path.
3. Execute the 'then' block if the condition is true.
4. Otherwise execute the 'else' block.
5. After the selected block, jump to the common continuation point if necessary.
Conceptually:
evaluate condition
branch to ELSE if condition is false
THEN block
jump to ENDIF
ELSE:
ELSE block
ENDIF:
next statement
Labels identify branch destinations. Thus, high-level selection is converted into explicit control-flow instructions.

Q6. [10 Marks] Explain how arrays are processed in MIPS programming.

Answer:
An array occupies consecutive memory locations. To access an element, the program needs the base address of the array and an offset corresponding to the element position.
A typical array-processing loop performs:
1. Keep the array base address in a register.
2. Maintain an index or byte offset.
3. Calculate the address of the required element.
4. Load the element using lw.
5. Perform arithmetic/logic processing in registers.
6. Store the result using sw if required.
7. Update the index/offset.
8. Branch back to the loop until the required number of elements has been processed.
For word arrays, offsets must account for the word size. Thus arrays combine memory addressing, register operations and loops.

5. Addressing Modes

Q1. [5 Marks] What is an addressing mode? Why is it needed?

Answer:
An addressing mode specifies how the CPU locates an operand or calculates the address used by an instruction.
It is needed because instructions may obtain operands from different places: registers, immediate fields, memory or control-flow targets.
The PPT covers register, immediate, base, PC-relative and pseudo-direct addressing. Each mode provides a compact way to specify where the required data or target address is located.

Q2. [5 Marks] Explain base addressing with its formula and purpose.

Answer:
Base addressing is used to access data in memory. The instruction contains a base register and an offset.
Formula:
Effective Address = Base Register + Sign-Extended Offset

The base register supplies the starting address, while the offset selects a location relative to that address.
It is especially useful for accessing arrays, structures and stack-relative data.

Q3. [8 Marks] Explain the major MIPS addressing modes covered in the PPT.

Answer:
1. Register addressing: the operands are located in registers.
2. Immediate addressing: a constant is contained directly in the instruction.
3. Base addressing: a memory address is calculated as Base Register + Offset.
4. PC-relative addressing: used by branch instructions; the target is calculated relative to the program counter.
5. Pseudo-direct addressing: used by j and jal instructions to specify a jump target.

The modes allow MIPS to represent different operand and control-flow locations within fixed instruction formats.

Q4. [8 Marks] A branch instruction has PC = 1000 and an offset of 5. Calculate its PC-relative branch target using the PPT formula.

Answer:
For PC-relative addressing, the PPT gives:
Branch Target = PC + 4 + (Offset × 4)

Given:
PC = 1000
Offset = 5

Step 1:
PC + 4 = 1004

Step 2:
Offset × 4 = 5 × 4 = 20

Step 3:
Target = 1004 + 20 = 1024

Therefore, the branch target address is 1024.
The multiplication by 4 accounts for the word-aligned MIPS instruction addresses.

Q5. [10 Marks] Compare base addressing, PC-relative addressing and pseudo-direct addressing with examples.

Answer:
Base addressing is used for memory operands. The effective address is:
EA = Base Register + Sign-Extended Offset.
Example form: lw $t0, offset($s0).

PC-relative addressing is used by branch instructions. Its target is:
Target = PC + 4 + (Offset × 4).
It is suitable for branches because the target is specified relative to the current code location.

Pseudo-direct addressing is used by jump instructions such as j and jal. Instead of calculating a memory data address, the instruction supplies a jump target field for transferring control.

Therefore:
• Base → memory/data address.
• PC-relative → branch target.
• Pseudo-direct → jump target.
These addressing modes make instruction encoding compact while supporting common processor operations.

Q6. [10 Marks] Solve an effective-address problem and explain the steps used in base addressing.

Answer:
Suppose a load instruction uses a base register containing 5000 and an offset of 24. Using:
Effective Address = Base Register + Sign-Extended Offset

Step 1: Base register = 5000.
Step 2: Offset = 24. It is sign-extended before addition; since it is positive, its value remains 24.
Step 3: Add:
EA = 5000 + 24 = 5024.

Therefore, the memory location accessed is address 5024.
If the offset were negative, it would represent a location before the base address. The important point is that the offset is combined with the base register to produce the effective memory address.

6. Parallelism

Q1. [5 Marks] Define latency, throughput and parallelism.

Answer:
Latency is the time required to complete one individual task from start to finish.
Throughput is the number of tasks completed per unit time.
Parallelism means executing multiple tasks simultaneously or overlapping their execution.
A key point from the PPT is that parallelism mainly improves throughput; it does not necessarily reduce the latency of one individual task.

Q2. [5 Marks] State the advantages and applications of parallelism.

Answer:
Advantages:
1. Higher overall throughput.
2. Better utilization of available hardware.
3. Higher performance without depending only on increased clock frequency.
Applications listed in the PPT include multicore processors, GPUs, supercomputers, digital signal processors and machine-learning accelerators.

Q3. [8 Marks] Explain spatial and temporal parallelism using the mathematical model in the PPT.

Answer:
For a task with latency L:
Without parallelism:
Throughput = 1/L.

Spatial parallelism:
If N hardware units execute independent tasks, then ideally:
Throughput = N/L.
The improvement comes from having multiple hardware units working at the same time.

Temporal parallelism:
If the task is divided into N stages and different tasks overlap in those stages, ideal throughput is also approximately:
Throughput = N/L.
Pipelining is the major example of temporal parallelism.
Thus, spatial parallelism uses more hardware, while temporal parallelism overlaps work across time.

Q4. [8 Marks] Explain the two-stage pipeline numerical example from the PPT.

Answer:
The PPT gives a circuit with an original critical path of 9.5 ns. Without pipelining:
Clock cycle time = 9.5 ns
Throughput = 1/9.5 ns ≈ 105 MHz.

After inserting a register, the two stages are:

Stage 1 = 5.5 ns
Stage 2 = 4.5 ns

The clock period is determined by the longest stage:
Tc = max(5.5, 4.5) = 5.5 ns.

Therefore:
Throughput = 1/5.5 ns ≈ 182 MHz.

Thus, pipelining increases the maximum operating frequency and throughput by dividing the long critical path into shorter stages.

Q5. [10 Marks] Explain pipeline hazards and their effect on processor performance.

Answer:
A pipeline hazard is a condition that prevents instructions from moving smoothly through the pipeline. Hazards can introduce stalls or bubbles and therefore reduce throughput.
1. Data hazard:
An instruction needs data that is still being produced by an earlier instruction. The dependent instruction may need to wait.
2. Control hazard:
Caused by branches or jumps. The processor may fetch instructions before the branch decision is known and may have to discard incorrect instructions.
3. Structural hazard:
Two or more instructions require the same hardware resource at the same time. One instruction must wait.
Therefore, although pipelining increases potential throughput, hazards prevent ideal performance. Pipeline design must handle these conditions to maintain efficient execution.

Q6. [10 Marks] A non-pipelined circuit has a clock period of 9.5 ns. A two-stage pipeline has stage delays of 5.5 ns and 4.5 ns. Calculate throughput and percentage throughput improvement.

Answer:
Without pipelining:
T = 9.5 ns
Throughput = 1/T = 1/9.5 ns ≈ 105.26 MHz.

With pipelining:
The clock period equals the longest stage:
T = max(5.5,4.5) = 5.5 ns.
Throughput = 1/5.5 ns ≈ 181.82 MHz.

Throughput improvement:
181.82 – 105.26 = 76.56 MHz.

Percentage improvement:
(76.56 / 105.26) × 100 ≈ 72.7%.

Therefore, the pipeline raises the maximum throughput from approximately 105 MHz to 182 MHz, an improvement of about 72.7%. The improvement occurs because the critical path per clock cycle is reduced.

7. Memory Arrays

Q1. [5 Marks] Explain the organization and size of a memory array.

Answer:
A memory array is made from many 1-bit storage cells arranged in rows and columns. If there are N address bits, the memory contains 2^N words. If each word contains M bits, the total size is:
Size = 2^N × M bits.
For example, with 10 address bits and 32-bit data:

2^10 = 1024 words
Size = 1024 × 32 = 32,768 bits = 32 Kb.
A decoder activates the wordline corresponding to the selected address.

Q2. [5 Marks] Compare DRAM and SRAM.

Answer:
DRAM stores each bit as charge on a capacitor. The charge leaks, so periodic refresh is required. A simplified DRAM cell uses about one transistor per bit, making it highly dense.
SRAM stores a bit using cross-coupled inverters. It does not need periodic refresh while power is applied and a typical cell uses six transistors per bit.
Therefore, DRAM is denser and generally slower, while SRAM is faster but requires more area per bit.

Q3. [8 Marks] Explain the read and write operations of a memory bit cell.

Answer:
Read operation:
1. The selected wordline is activated.
2. The selected cell is connected to its bitline.
3. The stored value influences the bitline.
4. The sense circuitry detects whether the stored value represents 0 or 1.

Write operation:
1. The desired data value is driven onto the bitline.
2. The wordline is activated to select the cell.
3. The driven bitline forces the cell to store the new value.
4. The wordline is then deactivated.
Thus, the wordline selects a cell while the bitline carries the data. The exact internal mechanism differs between SRAM and DRAM.

Q4. [8 Marks] Explain the area and delay trade-off among flip-flops, SRAM and DRAM.

Answer:
The PPT compares volatile storage technologies using transistor count and relative speed:
• Flip-flop: about 20 transistors/bit; very fast and the output is immediately available.
• SRAM: about 6 transistors/bit; faster than DRAM and does not require refresh while powered.
• DRAM: about 1 transistor/bit; very dense but slower and requires refresh.
More transistors generally mean more area, power and cost. Therefore, reducing the number of transistors improves density, but the resulting memory cell may have slower access or additional requirements such as refresh.

Q5. [10 Marks] A memory has 12 address bits and a 16-bit word. Calculate the number of words and total memory size. Explain the organization.

Answer:
Number of addressable words:
2^N = 2^12 = 4096 words.

Each word = 16 bits.

Total size:
4096 × 16 = 65,536 bits.

In bytes:
65,536 / 8 = 8,192 bytes = 8 KB.

Therefore, the memory organization is:
4096 words × 16 bits/word,
or 65,536 bits (8 KB).

The 12 address bits select one of the 4096 rows/words. The 16 data bits represent the word width. A decoder uses the address to activate the corresponding wordline, and the data bits are transferred through the bitlines.

Q6. [10 Marks] Explain how a 2^N-word × M-bit RAM can be described in HDL, including read and write operations.

Answer:
A RAM of depth 2^N and width M contains 2^N memory locations, each M bits wide. Conceptually it has address, data input, data output, clock and write-enable signals.
For a synchronous write, the write occurs on a clock edge when WE is asserted:

```
always_ff @(posedge clk)
if (we)
mem[adr] <= din;
```

This means:
1. Wait for the rising edge of the clock.
2. Check write enable.
3. If WE=1, store din at the selected address.

For an immediate/asynchronous read, the addressed location can be connected to the output:

```
assign dout = mem[adr];
```

For example, N=6 and M=32 gives 2^6 = 64 words, each 32 bits wide. Hence the RAM is 64 × 32 bits.

8. Sequential Building Blocks

Q1. [5 Marks] Define an N-bit binary counter and explain its operation.

Answer:
An N-bit binary counter is a sequential arithmetic circuit with a clock, reset and N-bit output Q.
When reset is asserted, Q is initialized to 0. After reset is released, the counter increments by 1 at every rising edge: Q_next = Q + 1.
For example:
0101 → 0110 → 0111 → 1000.
When the counter reaches its maximum value, the next increment wraps around to zero.
Counters are used for timing, event counting and frequency division.

Q2. [5 Marks] Explain a shift register and list its applications.

Answer:
A shift register stores multiple bits using a chain of flip-flops. On each clock edge, the stored bits move from one flip-flop to the next.
A serial input Sin introduces a new bit, while the last stored bit can leave through Sout.
Applications include:
1. Serial-to-parallel data conversion.
2. Parallel-to-serial data conversion.
3. Digital testing.
4. Data movement and temporary storage.
A parallel-load version can load multiple bits at once using additional control logic.

Q3. [8 Marks] Compare asynchronous and synchronous counters.

Answer:
Asynchronous/ripple counter:
• The first flip-flop receives the main clock.
• Subsequent flip-flops are triggered by earlier flip-flop outputs.
• Changes ripple through the counter.
• Propagation delays accumulate.
• It is simpler but limited in speed.

Synchronous counter:
• All flip-flops receive the same global clock.
• Outputs change in parallel.
• It does not suffer from cumulative ripple delay.
• It can operate at higher frequencies.
• It is also called a parallel counter.
Therefore, synchronous counters are preferred when higher-speed counting is required.

Q4. [8 Marks] Explain ring and Johnson counters and calculate their number of states.

Answer:
Ring counter:
A ring counter is a shift-register-based counter in which the last flip-flop output is fed back to the first. A single 1 circulates through the flip-flops.
For n flip-flops:
Number of valid states = n.
Example: a 4-bit ring counter can follow:
$1000 \rightarrow 0100 \rightarrow 0010 \rightarrow 0001 \rightarrow 1000$.

Johnson counter:
The complemented output of the last flip-flop is fed back to the first.
For n flip-flops:
Number of valid states = 2n.
Therefore, a 4-bit Johnson counter has:
$2 \times 4 = 8$ states.

Q5. [10 Marks] Explain the design steps for a synchronous counter using flip-flops.

Answer:
The PPT gives a systematic design procedure:
1. Determine the required counting sequence and number of states.
2. Determine the number of flip-flops required.
3. Draw the state diagram.
4. Prepare the present-state/next-state table.
5. Select the flip-flop type and use its excitation table.
6. Determine the required inputs to each flip-flop.
7. Simplify the input equations using Boolean algebra/K-maps.
8. Implement the resulting combinational logic.
9. Connect all flip-flops to the common clock.
10. Verify that the required sequence is produced.
The key feature is that all flip-flops receive the same clock, so state transitions occur synchronously.

Q6. [10 Marks] Explain scan-chain testing of sequential circuits and the 32-bit counter example.

Answer:
Sequential circuits are harder to test than combinational circuits because their outputs depend on internal state. Testing a particular internal state directly could require an extremely large number of clock cycles.
A scan chain solves this problem by connecting the flip-flops into one long shift register.
Normal mode:
• Each flip-flop receives its normal D input.
Test mode:
• Data is shifted serially through Sin and Sout.
• Internal state can be loaded directly.
• Internal state can also be shifted out and observed.

For the 32-bit counter example in the PPT:
1. Shift a 32-bit test pattern into the scan chain $\rightarrow$ 32 cycles.
2. Allow the counter to operate for one clock cycle $\rightarrow$ 1 cycle.

3. Shift the 32-bit result out $\rightarrow$ 32 cycles.
Total = 32 + 1 + 32 = 65 cycles.
This is dramatically smaller than testing by simply waiting for approximately 2 billion counter transitions.